Practical - 9

Enrollment No.: 250843116002

Aim: Design and develop a simple interactive dashboard using Dash/Plotly for business reporting.

Step 1: Install libraries

```
# Install the required libraries
!pip install -q dash plotly pandas
```

Step 2: Import Required libraries

```
# Import Pandas
import pandas as pd
# Import Plotly Express
import plotly.express as px
# Import Dash components
from dash import Dash, dcc, html, Input, Output
```

Step 3: Create the dataset

```
# Create a  dataset
data = {
    "Product": ["Laptop", "Mobile", "Tablet", "Laptop", "Mobile",
                "Tablet", "Laptop", "Mobile", "Tablet", "Laptop"],

    "Region": ["North", "South", "East", "West", "North",
               "South", "East", "West", "South", "North"],

    "Sales": [50000, 30000, 20000, 45000, 35000,
              25000, 60000, 40000, 22000, 55000],

    "Profit": [10000, 6000, 4000, 9000, 7000,
               5000, 12000, 8000, 4400, 11000]
}

df = pd.DataFrame(data)

print(df)
```

```
  Product Region  Sales  Profit
0  Laptop  North  50000   10000
1  Mobile  South  30000    6000
2  Tablet   East  20000    4000
3  Laptop   West  45000    9000
4  Mobile  North  35000    7000
5  Tablet  South  25000    5000
6  Laptop   East  60000   12000
7  Mobile   West  40000    8000
8  Tablet  South  22000    4400
9  Laptop  North  55000   11000
```

Step 4: Create the dashboard layout

```
# Create a Dash application
app = Dash(__name__)

# Design the dashboard
app.layout = html.Div([

    # Add title
    html.H1(
        "Business Reporting Dashboard",
        style={"textAlign": "center", "color": "blue"}
    ),

    # Add a label
    html.Label("Select Region:"),

    # Create a dropdown
    dcc.Dropdown(
```

```
        id="region",
        options=[
            {"label": "All Regions", "value": "All"}
        ] + [
            {"label": r, "value": r}
            for r in sorted(df["Region"].unique())
        ],
        value="All",
        clearable=False
    ),

    html.Div(id="summary"),

    dcc.Graph(id="sales_chart"),

    dcc.Graph(id="profit_chart"),

    dcc.Graph(id="product_chart")

], style={"padding": "20px"})
```

Step 5: Add interactive dashboard functionality

```
# Update the dashboard
@app.callback(
    Output("summary", "children"),
    Output("sales_chart", "figure"),
    Output("profit_chart", "figure"),
    Output("product_chart", "figure"),
    Input("region", "value")
)

# Define the function
def update_dashboard(region):

    # Filter the dataset
    if region == "All":
        filtered = df
    else:
        filtered = df[df["Region"] == region]

    # Calculate total sales and profit
    total_sales = filtered["Sales"].sum()
    total_profit = filtered["Profit"].sum()

    # Display the summary information
    summary = html.Div([
        html.H3(f"Total Sales: ₹{total_sales:,}"),
        html.H3(f"Total Profit: ₹{total_profit:,}"),
        html.H3(f"Total Records: {len(filtered)}")
    ])

    # Create a bar chart
    sales_fig = px.bar(
        filtered,
        x="Product",
        y="Sales",
        color="Region",
        title="Sales by Product"
    )

    # Create a pie chart
    profit_fig = px.pie(
        filtered,
        names="Region",
        values="Profit",
        title="Profit Distribution by Region"
    )

    # Calculate total profit
    product_data = filtered.groupby(
        "Product", as_index=False
    )["Profit"].sum()

    # Create a bar chart
    product_fig = px.bar(
        product_data,
        x="Product",
        y="Profit",
```

```
        color="Product",
        title="Profit by Product"
    )


    return summary, sales_fig, profit_fig, product_fig
```

Step 6: Run the dashboard in Google Colab

```
# Run the dashboard
app.run(jupyter_mode="inline", debug=False, port=8050)
```

